

Query Service

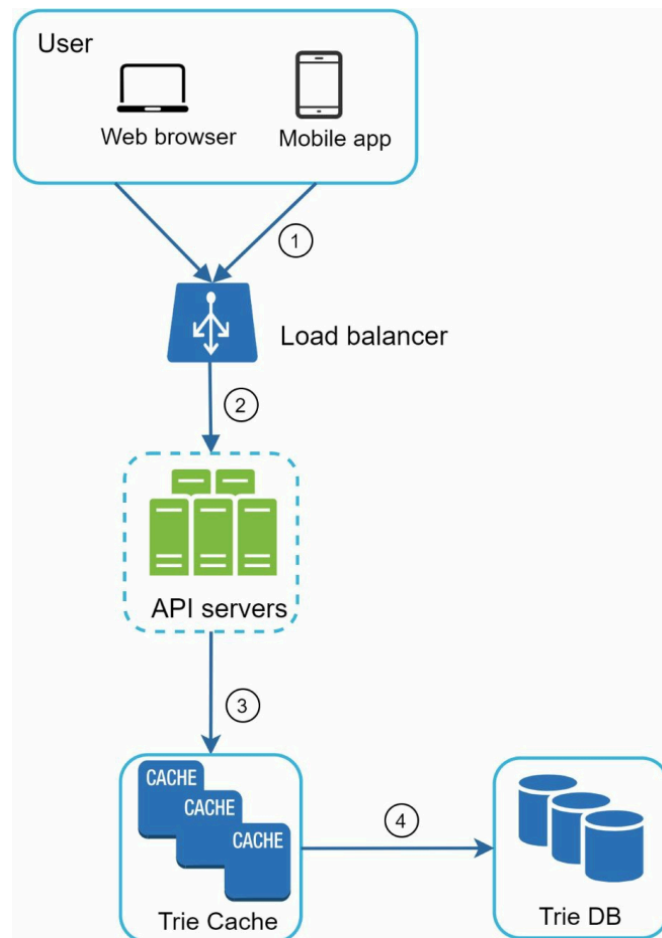
In the earlier high-level design, the query service directly queried the database to retrieve the top 5 autocomplete suggestions.

This design is inefficient because:

- Database queries are expensive
- Prefix searches are slow at scale
- Sorting large datasets increases latency
- High traffic can overload the database

To achieve fast autocomplete responses, we improve the architecture using caching.

Figure shows the improved query service design.



Improved Query Service Flow

The optimized query service works as follows:

Step 1: Client Sends Search Query

A user types characters into the search box.

Example:

None
`search?q=sy`

The request is sent to the load balancer.

Step 2: Load Balancer Routes Request

The load balancer distributes incoming requests across API servers.

Benefits:

- High availability
- Horizontal scalability
- Better traffic distribution

Step 3: API Servers Fetch Trie Data

API servers retrieve autocomplete suggestions from the Trie Cache.

The cache already stores:

- Prefix mappings
- Top k search queries

Example:

None

Prefix: "sy"

Suggestions:

1. system design
2. syntax
3. system programming
4. symphony
5. symbols

API servers then construct the response and send it back to the client.

Step 4: Cache Miss Handling

Sometimes requested trie data may not exist in the cache.

Reasons:

- Cache server restart
- Memory eviction
- Cache node failure
- Insufficient cache space

In this case:

1. API server fetches trie data from Trie DB.
2. Data is reloaded into Trie Cache.
3. Response is returned to the client.

This mechanism is called:

None

Cache Replenishment

Subsequent requests for the same prefix are served directly from the cache.

Why Trie Cache is Important

Advantages of Trie Cache:

Benefit	Description
Fast lookup	In-memory retrieval
Low latency	Millisecond response
Reduced DB load	Fewer database queries
Better scalability	Handles high QPS

Since autocomplete systems require responses within:

None

100 ms

cache becomes a critical component.

Query Service Optimizations

To further improve performance, several optimizations are added.

1. AJAX Requests

For web applications, autocomplete suggestions are usually fetched using AJAX.

AJAX = Asynchronous JavaScript and XML

How AJAX Helps

Without AJAX:

- Entire page reloads after every request

With AJAX:

- Only autocomplete data is fetched
- Web page remains unchanged

Benefits:

- Faster user experience
- Reduced bandwidth usage
- Smooth typing interaction

Example AJAX Flow

User types:

None
sys

Browser sends asynchronous request:

None
GET /autocomplete?q=sys

Server returns suggestions without refreshing the page.

2. Browser Caching

Autocomplete suggestions often remain unchanged for short periods.

Therefore:

- Browser can cache autocomplete responses locally.

This avoids repeated server requests for the same prefix.

Benefits of Browser Cache

Benefit	Description
Faster response	Local retrieval
Reduced server traffic	Fewer API calls
Lower latency	No network delay

Example Cache-Control Header

Google Search uses browser caching.

Example:

None

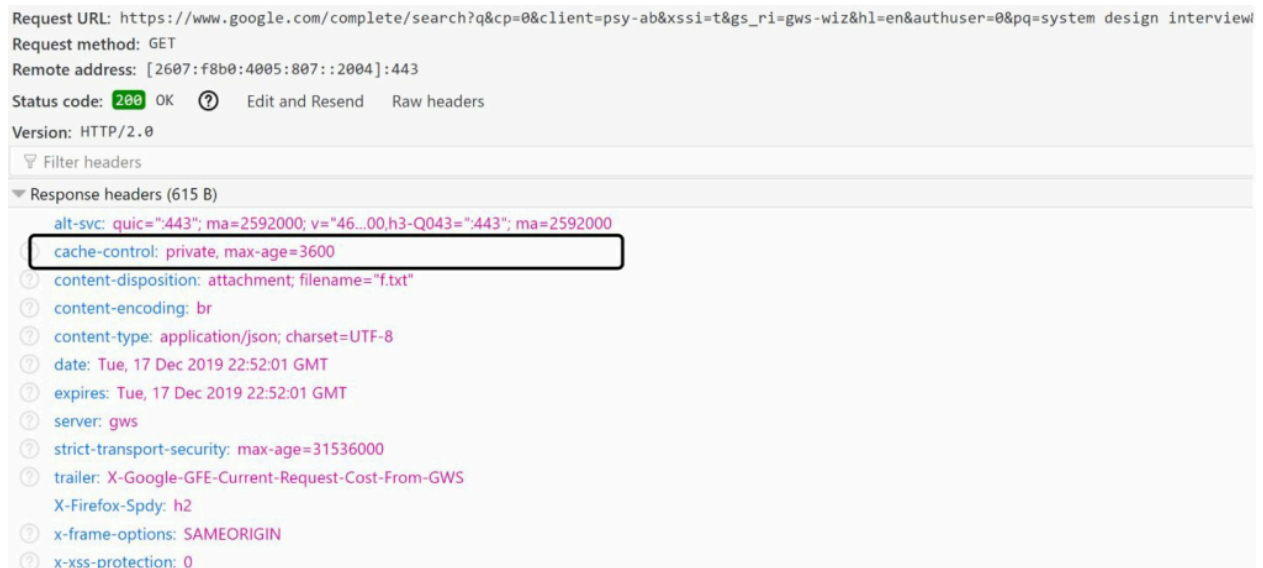
Cache-Control: private, max-age=3600

Meaning:

- **private** → cache is user-specific
- **max-age=3600** → valid for 1 hour

Thus, repeated autocomplete queries within one hour can be served directly from browser cache.

Figure shows an example response header.



3. Data Sampling

At very large scale, logging every search query is expensive.

Problems:

- Huge storage requirements
- Increased processing cost
- High write throughput

To reduce overhead, data sampling is used.

Data Sampling Technique

Instead of logging every request:

- Only a subset is logged.

Example:

None

Log 1 out of every 100 requests

This significantly reduces:

- Storage consumption
- Processing cost
- Analytics overhead

while still maintaining statistically useful data.

Overall Query Flow

Step 1

User types a search prefix.

Step 2

Browser sends AJAX request.

Step 3

Load balancer routes request.

Step 4

API server checks Trie Cache.

Step 5

If cache hit:

- Return top k suggestions immediately.

Step 6

If cache miss:

- Fetch from Trie DB
- Replenish cache
- Return suggestions

Step 7

Browser caches response locally.

Query Service Architecture Components

Component	Responsibility
Load Balancer	Traffic distribution
API Servers	Handle autocomplete requests
Trie Cache	Fast prefix lookup

Trie DB	Persistent trie storage
Browser Cache	Local response caching

Advantages of Improved Query Service

Optimization	Benefit
Trie Cache	Fast in-memory lookup
AJAX	No page reload
Browser Cache	Reduced latency
Cache Replenishment	Better fault tolerance
Data Sampling	Reduced storage cost

Final Design Goals Achieved

The optimized query service satisfies all major requirements:

Requirement	Achieved By
Fast response time	Trie cache
Scalability	Load balancing + caching
High availability	Distributed cache
Low latency	AJAX + browser cache
Reduced cost	Data sampling

This architecture enables the autocomplete system to handle:

- High QPS
- Large datasets
- Low latency requirements
- Millions of daily active users efficiently.